

Claude Code for Academic Researchers

Course Notes — Sessions 1 & 2 | Brady Allardice, UPF / IBEI

1. The Core Idea

Claude Code is an agentic AI coding tool that runs inside your terminal. Unlike a chatbot, it can read, write, and execute files directly inside your project folder.

Central principle: *You are the Principal Investigator. Claude Code is the Research Assistant. You direct the work, make analytical decisions, and verify every result. Claude Code handles the mechanical execution.*

- **Strength:** It is extremely fast and has read a lot.
- **Risk:** It will never say 'I don't know how to do that' — it will do it wrong and hand it in.
- **Key question:** The question is never 'can Claude Code do this?' It almost always can. The question is: what is the most reliable way to get it done?

2. The Coding Paradigm Spectrum

Claude Code sits at the 'agentic coding' end of a spectrum:

- Manual Coding — you write every line (what you do now in R/Python)
- Autocomplete / Copilot — AI suggests the next line; you accept or reject
- Web Chat — you describe a task; AI gives code to copy-paste; AI cannot see your project
- Agentic Coding — AI reads, writes, and runs code inside your actual project

Key insight: *Agentic coding is the first paradigm where the tool interacts with your project directly and autonomously. That is what makes it so productive — and why learning to use it well matters.*

3. How Claude Code Works

File-based, not interactive

- In R/Python you open a session, load data into memory, and run commands interactively.
- Claude Code reads and writes files on disk. It has no persistent session with your data loaded.
- Workflow: reads your files → writes/modifies scripts → optionally runs them.
- Think in projects and file paths, not in interactive exploration.

The Context Window

- Claude Code can only 'see' a limited amount of text at once: your instructions + files it reads + its own responses.
- In long conversations or large projects, it loses earlier context and may introduce inconsistencies between files.
- Keep tasks focused. Start a fresh conversation for each new task.

Execution Modes

The most important configuration decision you make each session:

- Plan Mode (recommended for research) — Claude Code proposes a full plan before acting. You review, approve, modify, or reject. This is where you catch 'I am going to do an inner join' before 300 observations disappear.
- Default Mode — acts autonomously, pauses when it judges something 'significant'. Its judgment may not match yours. Use with caution.
- Auto-Accept — runs everything without asking. Only for low-risk tasks like reformatting or documentation. Never for your data.

Key insight: *Plan mode = you direct the work. Auto-accept = you review after the fact. For published research, directing is almost always safer.*

Model Selection

- Haiku — fast, cheap. Good for mechanical tasks: renaming variables, reformatting files.
- Sonnet — everyday workhorse. Data cleaning, building panels, debugging, documentation.
- Opus — maximum reasoning. Complex multi-file refactors, analytical code review. Uses ~5x more tokens than Haiku.
- Rule of thumb: default to Sonnet. Upgrade to Opus when correctness really matters. A wrong answer fast is worse than a right answer slow.

4. Key Tools & Configuration

CLAUDE.md

A plain text file in your project folder that Claude Code reads automatically at the start of every session. The single most effective way to prevent errors before they happen.

What to include:

- Project description and research question
- Data structure: file names, key variables, units, expected ranges
- Coding conventions (e.g. always use robust standard errors)
- Hard constraints: 'Never drop observations without logging count and reason'
- 'All merges must report pre- and post-merge row counts'
- What Claude Code should NOT do (e.g. do not choose model specifications)

.claudeignore

Hides sensitive files from Claude Code (IRB-protected data, credentials). Create a plain text file named .claudeignore in your project folder. Add one pattern per line. Set this up before your first session.

Scripts (.py) vs Notebooks (.ipynb)

- Claude Code operates on files, not interactive sessions. Use .py scripts.
- Scripts run end-to-end: `python my_script.py` always does the same thing — exactly what reproducible research needs.
- This means more checkpointing, more logging (print row counts, shapes), and more intermediate file saves between pipeline stages.

5. Verification & The Control Principle

Code that runs without errors can still produce completely wrong results. These are the errors that end up in published work:

- Silent observation loss — a merge silently drops 300 observations; the code looks clean.
- Invalid transformation — a log applied to a variable containing zeros; no error raised.

- Incorrect panel operations — a lagged variable computed without sorting by time within panels.

The control principle: *If you cannot verify a result, you should not delegate the task. Every decision in the code is yours. You must be able to justify it as if you chose it yourself.*

Before you delegate: ask 'What would wrong look like? Can I describe a specific failure mode?'

After you delegate: 'Can I describe what this code does without reading it? Can I state what decisions shaped the output and why?'

Setup Checklist — 8 Steps to Get Started

Complete before Session 3 | Apr 17, 11am

Open your terminal and run each command to verify. Tick each step as you complete it.

#	What to do	Command / Action
1	Install VS Code (if not done)	Download from code.visualstudio.com — free editor with built-in terminal
2	Install Node.js	Download from nodejs.org → verify: <code>node --version</code>
3	Install Claude Code	<code>npm install -g @anthropic-ai/claude-code</code>
4	Verify Claude Code installed	<code>claude --version</code> (should show a version number)
5	Clone the course repository	<code>git clone https://github.com/bradyallardice/claude-code-workshop.git</code>
6	Open the folder in VS Code	File → Open Folder → select <code>claude-code-workshop</code>
7	Create a virtual environment	<code>python3 -m venv AIAgents</code> then <code>source AIAgents/bin/activate</code> (Mac/Linux) or <code>AIAgents\Scripts\activate</code> (Windows)
8	Launch Claude Code to test	<code>cd claude-code-workshop</code> then <code>claude</code> → type: What files are in this directory?

Already cloned the repo? Run `git pull` inside the folder to get the latest materials before tomorrow.

What Session 3 covers (Friday Apr 17)

- Module 5: Code Auditing and Review — using Claude Code to check existing scripts for silent errors, verify code matches your methods section, catch observation loss and transformation mistakes.
- Module 6: Version Control with Git — committing your project state before each Claude Code session so you can always roll back. Core commands: `git add`, `git commit`, `git diff`.

Your advantage going in

Your R and Python background means you already know what 'wrong' looks like in a data pipeline — you check merges, verify row counts, sanity-check distributions. That is exactly the skill the auditing module builds on. Claude Code generates the code; your domain knowledge is what catches the errors.